BIG DATA · UDEM

Big Data e Ingeniería de Datos

Clase 1 · ¿La lluvia cambia los viajes en NYC?

Posgrado · UdeM · 3 nov 2026

En 8 sesiones construyes un sistema de datos de punta a punta sobre los viajes de NYC

Arquitectura

– Warehouse, lake, lakehouse, medallion

Procesamiento

– SQL en Snowflake, Spark en Glue, Airflow

Operación

– Git, CI/CD, infraestructura como código, calidad y costos

Las 8 sesiones van de la arquitectura al pipeline en producción

Fecha	Sesión
3 nov	Ingeniería de datos y arquitectura
4 nov	Cómo se lee big data · SQL en Snowflake
9 nov	SQL avanzado y modelado dimensional
10 nov	Del lake a Snowflake · lanza Taller 1
17 nov	Spark, pruebas y Git
18 nov	Airflow y Glue hacia Snowflake · lanza Taller 2
23 nov	Rendimiento, costos, calidad y producción
24 nov	Trabajo de Taller 2 y cierre

La nota sale de dos talleres en equipos de máximo 4 personas

Taller 1 · 50 %

– Modelo estrella y preguntas de negocio en SQL

– Viajes reales de NYC en Snowflake

– Lanza 10 nov · **entrega 15 nov**

Taller 2 · 50 %

– Pipeline Spark, Glue y Airflow → Snowflake

– Con pruebas, CI y costos

– Lanza 18 nov · **entrega 29 nov**

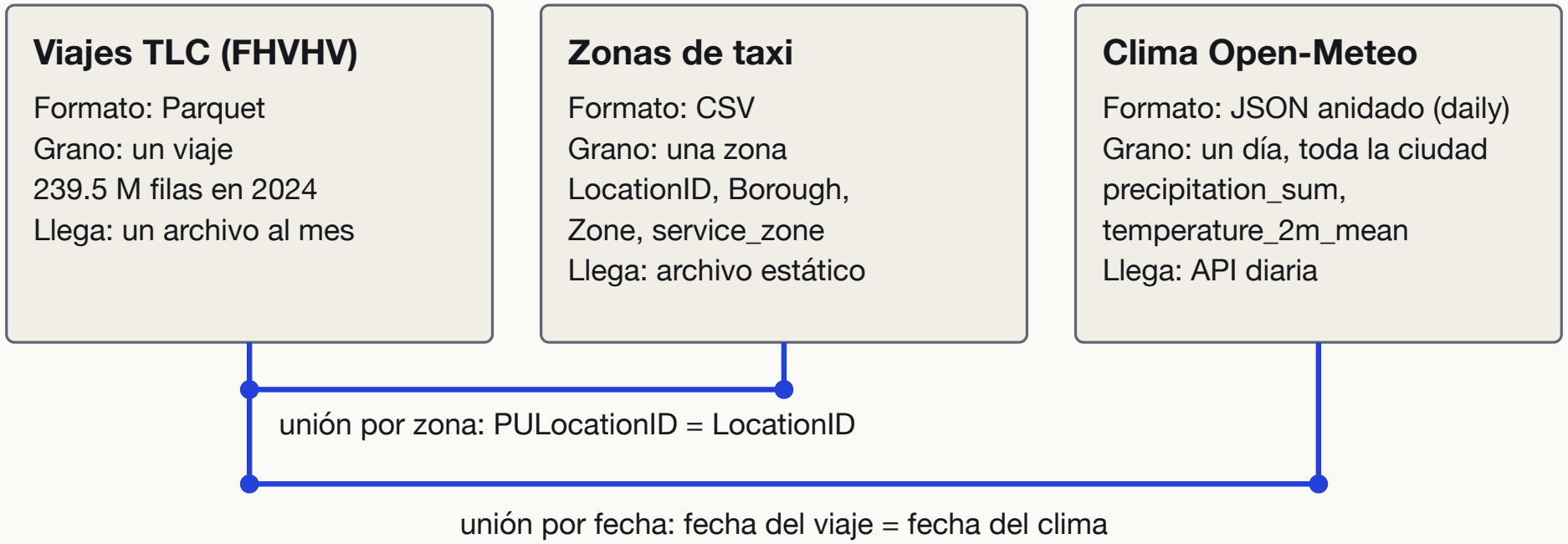
EL CASO QUE ATRAVIESA EL CURSO

¿La lluvia cambia los viajes en NYC?

La TLC (Comisión de Taxis y Limusinas de NYC, el regulador) decide política de movilidad

Responder exige cruzar tres fuentes sin formato, grano ni cadencia comunes

Tres fuentes, tres formatos, tres granos: ninguna responde sola.



Grano = qué representa una fila · TLC = Comisión de Taxis y Limusinas · FHVHV = viajes de plataformas como Uber y Lyft

239.5 M

Excel admite 1 048 576 filas: el año son ~228 hojas. 5.4 GB caben en un portátil: lo difícil son 3 fuentes, la repetición mensual y la confiabilidad.

NYC TLC · 2024

Cada pieza del curso cumple un requisito de la pregunta



Costos, calidad y producción: clase 7. Taller 2: clase 8.

S3 = almacenamiento de archivos · Spark = motor de procesamiento distribuido · Glue = Spark administrado por AWS · Airflow = orquestador · Snowflake = warehouse en la nube

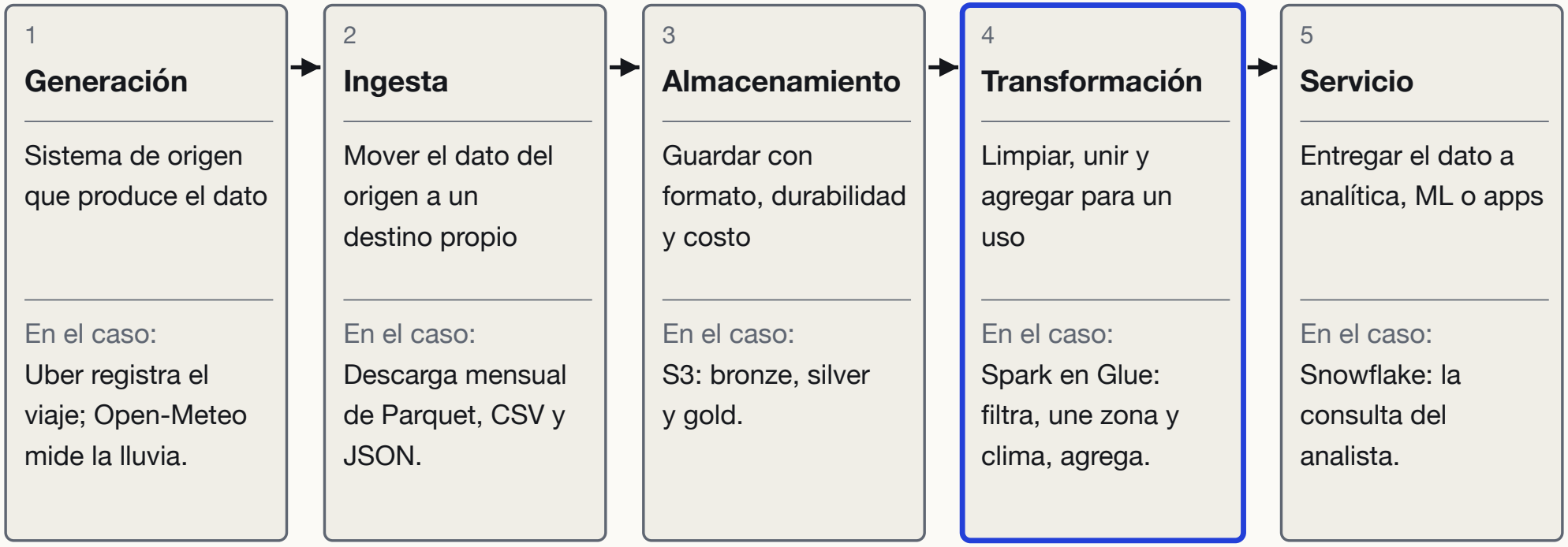
01

INGENIERÍA DE DATOS

¿Qué sistema lleva el viaje que registra la app hasta la decisión de la TLC, el regulador?

La ingeniería de datos diseña y opera los sistemas que llevan el dato del origen al uso

Ciclo de vida: las cinco etapas por las que pasa un dato, desde que se genera hasta que alguien lo usa.



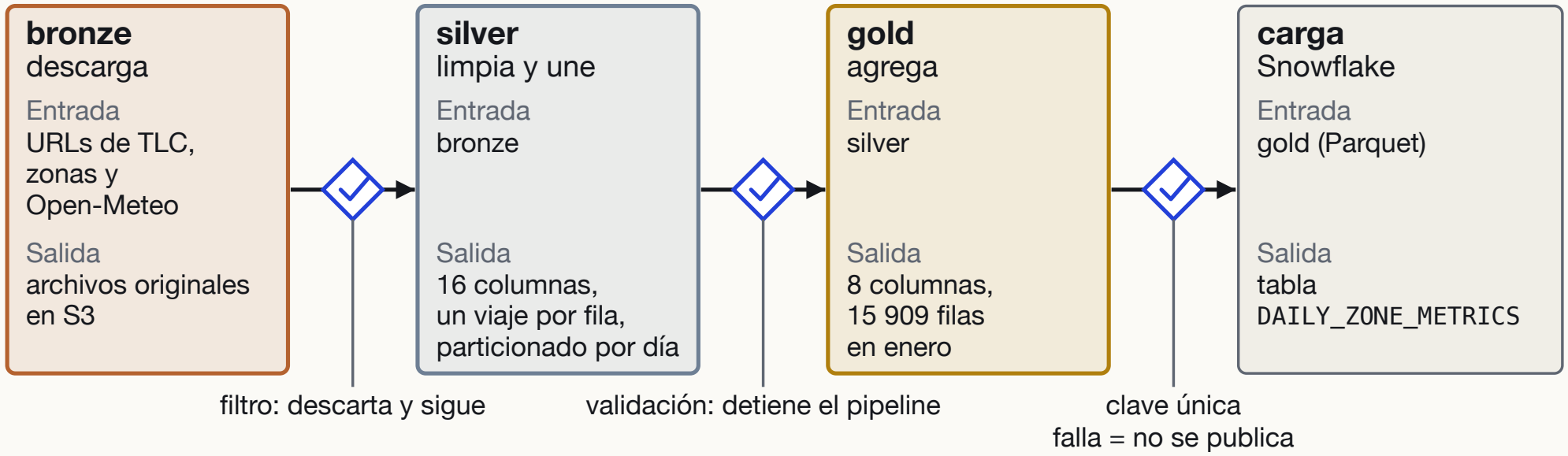
Transversales: seguridad, gestión de datos, DataOps, arquitectura, orquestación, software.

Tres roles, un mismo dato, tres preguntas distintas

Rol	Pregunta	En el caso
Ingeniería de datos	¿Llega completo y reproducible?	Pipeline que produce la tabla diaria
Analista	¿Qué pasó y cuánto?	Lluvia contra seco
Ciencia de datos	¿Qué pasará?	Demanda según clima

Un pipeline encadena pasos con entrada y salida definidas hasta producir un conjunto de datos consumible

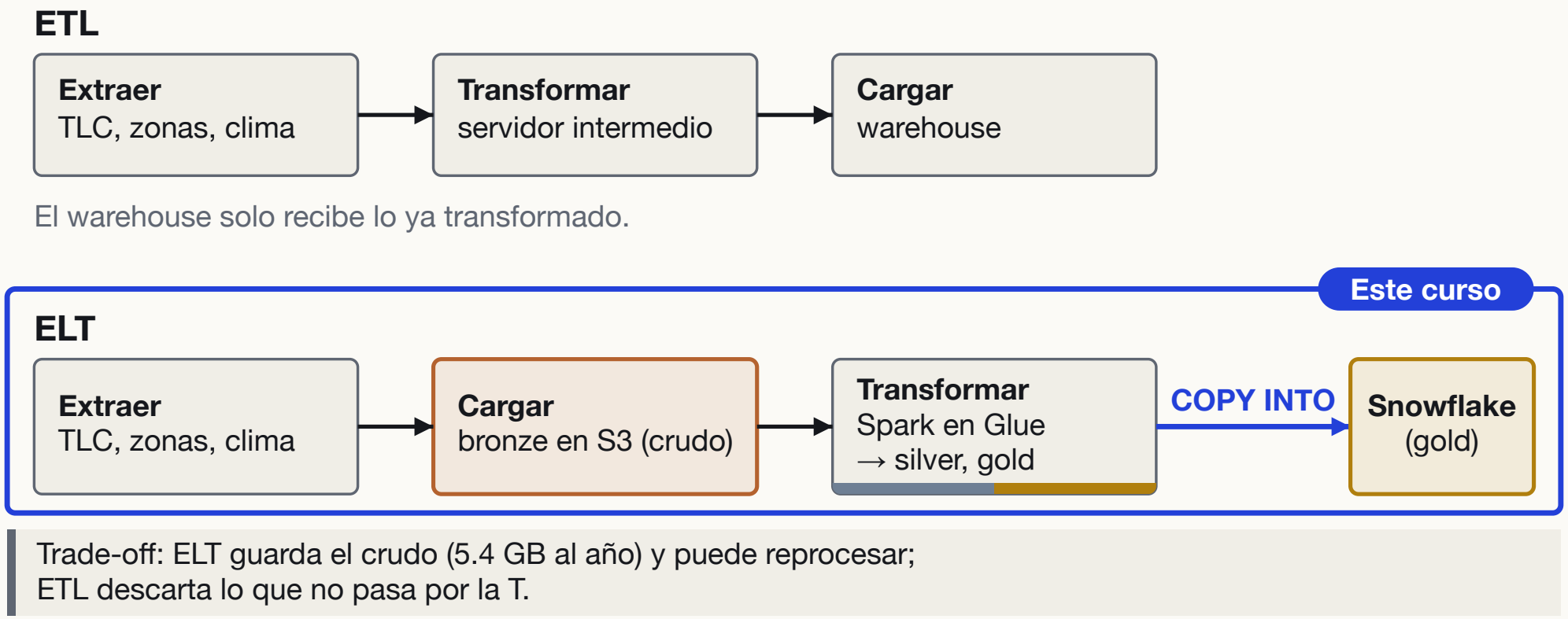
Pipeline: secuencia de pasos donde la salida de uno es la entrada del siguiente; cada paso es verificable y repetible.



bronze = tal como llegó · silver = limpio y unido · gold = agregado para responder.

El filtro descarta filas; la validación y la clave única detienen el pipeline y no se publica

ETL transforma antes de cargar; ELT carga el crudo y transforma después

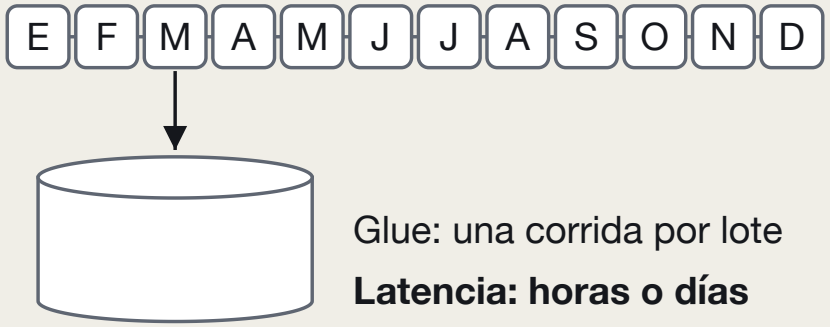


> Trade-off: ELT guarda el crudo (5.4 GB al año) y puede reprocesar; ETL descarta lo que no pasa por la T.

El curso hace ELT sobre el lake y carga a Snowflake solo gold, ya transformado.

Este caso es batch: los datos llegan en archivos mensuales y la pregunta es histórica

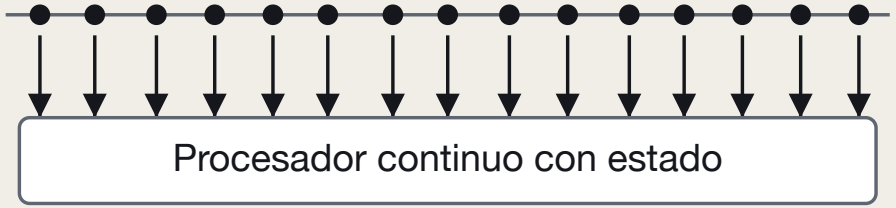
Batch — **Este caso**



Glue: una corrida por lote

Latencia: horas o días

Streaming



Latencia: segundos

Maneja: orden, eventos tardíos, estado

> Streaming se justificaría para despacho o tarifa dinámica; no para comparar 2024 consigo mismo.

Streaming procesa evento a evento, con latencia de segundos y más complejidad.

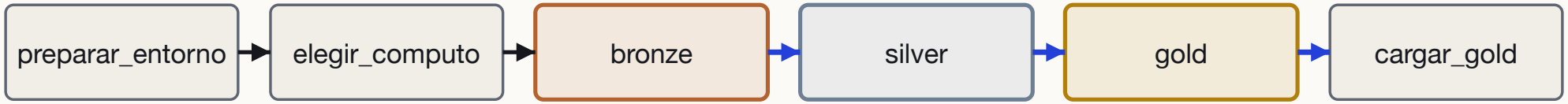
Un orquestador ejecuta el pipeline como un DAG: dependencias explícitas, estado por tarea y reproceso por mes

> DAG: grafo dirigido sin ciclos de tareas; el orquestador respeta las dependencias, registra el estado y permite reejecutar.

retries: configurable por tarea (aquí 0: el fallo queda visible)

params: mes = 2024-01
computo = glue_large

DELETE del mes + COPY INTO



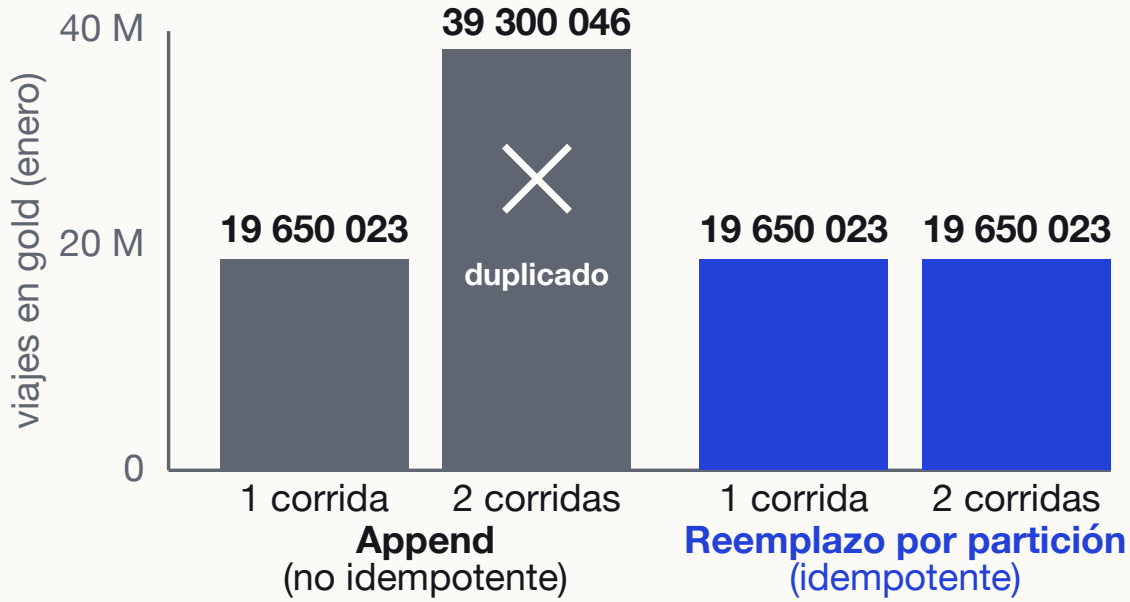
dependencia: silver no inicia si bronze falla

En dev, el mismo DAG corre en local sobre una muestra.

> Reproceso de enero: lanzar el DAG con mes = 2024-01; los demás meses no se tocan.

Airflow define cada paso como tarea y su orden; el cómputo lo hace Glue.

Un paso idempotente deja el mismo resultado si se ejecuta una o diez veces



silver: borra pickup_date=2024-01-*
y reescribe

gold: borra mes=2024-01 y reescribe

Snowflake: DELETE del mes
+ COPY INTO (FORCE = TRUE)

> Idempotente no es atómico, en S3: entre el borrado y la escritura el mes queda vacío;
> si la escritura falla, sigue vacío hasta relanzar. Lo resuelve un formato de tabla (Iceberg, Delta).

> Dato derivado: 39 300 046 = 2 × 19 650 023 (suma real de viajes de enero).

A

PARADA: GITHUB Y SNOWFLAKE

¿Dónde vive el código del pipeline y dónde se servirá gold?

Tu fork versiona el pipeline; Actions lo despliega

GitHub

1. Crea la cuenta: github.com > Sign up.
2. Abre github.com/Diegomanzur10/bigdata-udem-2026.
3. Fork > Create fork: tu copia es TU-USUARIO/bigdata-udem-2026.
4. Actions > "I understand my workflows…" > enable.
5. Debes ver: Actions con workflows habilitados.
6. Guía: curso/clase01/cuentas.md

El trial de Snowflake es donde se servirá gold

Snowsight

1. signup.snowflake.com: crea el trial.
2. AWS, US East (Ohio), Enterprise.
3. Activa por correo; guarda usuario y URL.
4. Projects > Worksheets > + SQL Worksheet.
5. Ejecuta `SELECT CURRENT_REGION();`
6. Debes ver `AWS_US_EAST_2`; si no, repítelo.

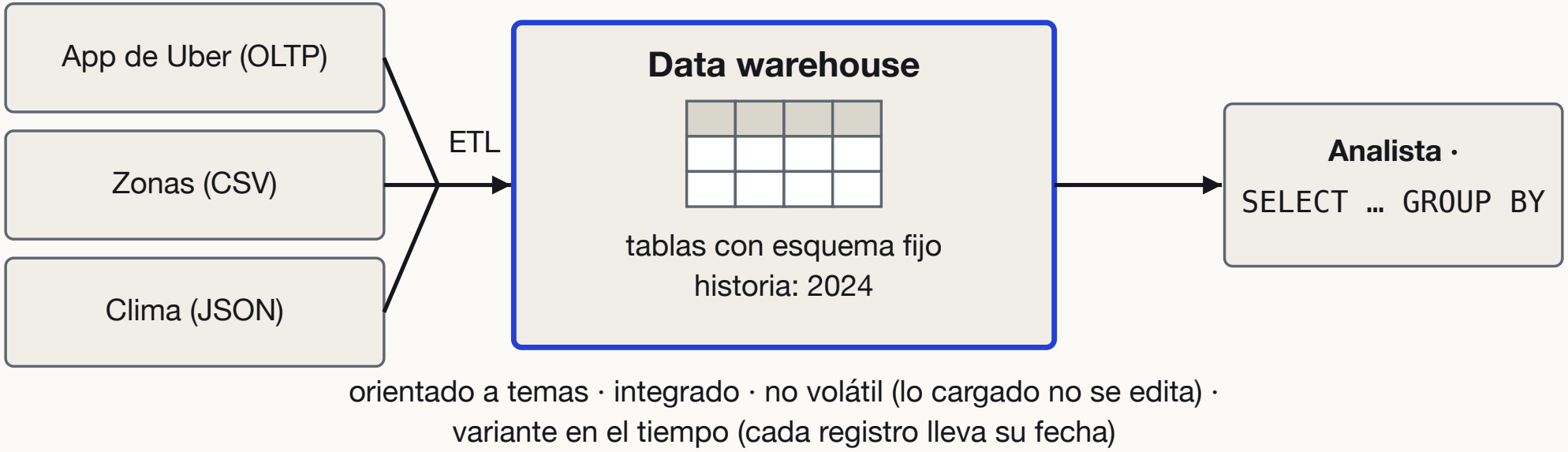
02

DATA WAREHOUSE

¿Dónde se consulta rápido lo que ya sabemos que queremos preguntar?

Un data warehouse integra varias fuentes en un almacén histórico optimizado para análisis

Data warehouse: base analítica que integra datos de varios sistemas, con historia, sin modificar lo ya cargado.



orientado a temas · integrado · no volátil (lo cargado no se edita) · variante en el tiempo (cada registro lleva su fecha)

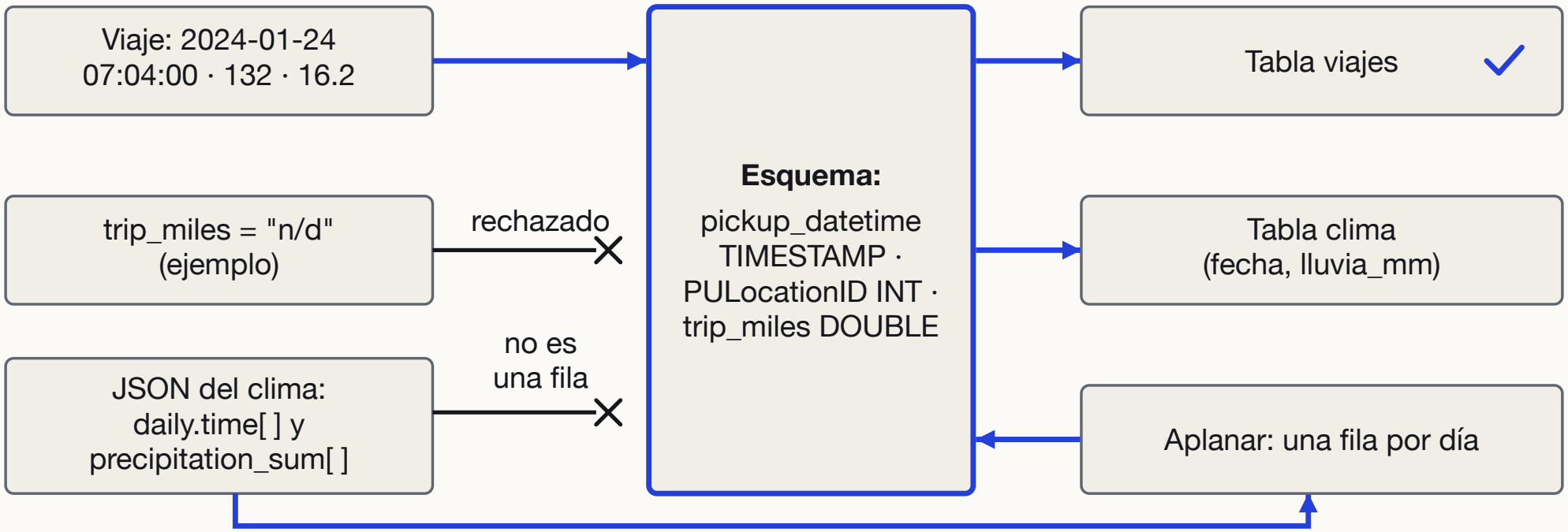
Separar el warehouse de los sistemas de origen evita que una consulta de meses degrade la app.

Esquema definido y SQL: el analista consulta sin tocar los sistemas de origen.

Uber registra un viaje (OLTP); el regulador agrega meses (OLAP)

Carga	OLTP (transaccional)	OLAP (analítico)
Operación	INSERT de 1 viaje	SUM por zona y día
Filas por consulta	1	millones
Latencia	milisegundos	segundos
Almacén	por fila	por columna
Modelo	normalizado	dimensional

Esquema al escribir: el warehouse valida tipos y estructura antes de guardar, y rechaza lo que no encaja



Límite: si TLC agrega una columna, en un warehouse clásico hay que alterar la tabla (ALTER TABLE ADD COLUMN); las filas viejas quedan en NULL.

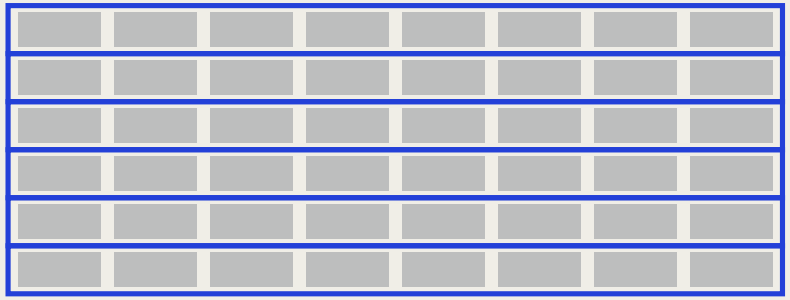
Consultas predecibles; a cambio, una columna nueva en la fuente exige migrar la tabla.

Un almacén columnar lee solo las columnas que la consulta usa y comprime cada columna por separado

```
SELECT PULocationID, AVG(tips) FROM viajes GROUP BY PULocationID
```
— usa 2 de las 24 columnas.

Por filas

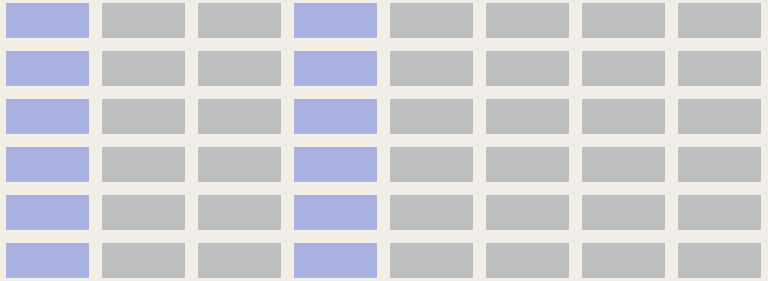
PU DO mi **tips** … … … …



Lee 24 de 24 columnas por cada viaje

Por columnas

PU DO mi **tips** … … … …



Lee 2 de 24 columnas

Mismo tipo contiguo: mejor compresión.

Límite: reconstruir o actualizar un viaje completo toca las 24 columnas.

El modelo estrella separa el hecho (el viaje) de las dimensiones que lo describen



En el curso, silver desnormaliza: borough, zona y lluvia_mm ya vienen unidos al viaje.

Grano: una fila de hechos = un viaje. Las medidas se suman; las dimensiones agrupan.

El grano fija qué pregunta responde una tabla

Grano	Filas (enero)	Responde
Un viaje	19.7 M	¿Cuánto duró este viaje?
Fecha × zona × empresa	15 909	¿Viajes de Uber en JFK por día?
Mes × empresa	2	¿Uber o Lyft?

Guardar todo el crudo en un warehouse obliga a modelar antes de saber qué columnas servirán

15 de 24 columnas usadas



¿Guardar las 9 restantes, con costo, o descartarlas, sin reprocesar?

```
{"daily": {
  "time": [...],
  "precipitation_sum": [...]}}
```

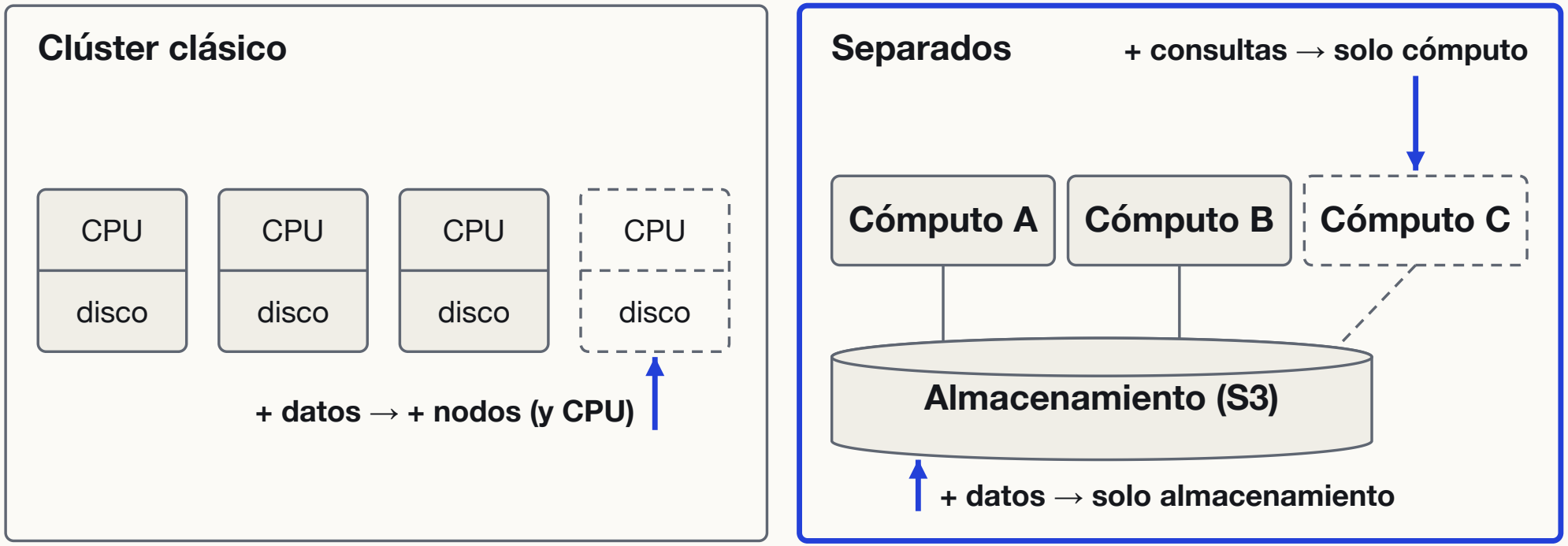
aplanar: una fila por día

fecha	lluvia_mm
2024-01-01	0.0
2024-01-02	0.0
2024-01-24	3.5

> O decides antes qué guardar, o pagas por conservar lo que quizá nunca uses.

Snowflake admite JSON (VARIANT), pero el crudo queda en el formato y el motor del proveedor.

En un warehouse clásico, cómputo y almacenamiento crecen juntos: más datos obligan a pagar más cómputo



> Bronze crece 5.4 GB al año; la consulta de la demo dura segundos: almacenamiento y cómputo crecen distinto.

Los motores en la nube separan ambos; la clase 2 muestra cómo lo hace Snowflake.

El warehouse resuelve la consulta; el lake resuelve guardar y reprocesar el crudo

Warehouse resuelve

– Consultas rápidas con SQL sobre datos modelados

– Esquema validado al escribir

Falta resolver

– Guardar los 3 orígenes tal como llegan, en archivos abiertos

– Reprocesar meses cuando cambia una regla

– Leer el JSON del clima sin modelarlo antes

DESCANSO

Si el warehouse pide decidir el esquema antes, ¿dónde guardamos el crudo tal como llegó?

Volvemos en 20 min.

B

PARADA: AWS

¿Dónde vivirán bronze, silver y gold, y quién ejecutará el pipeline?

Tu cuenta de AWS aloja S3 (lake), Glue (cómputo) y Airflow (orquestación)

Consola AWS

1. Abre el registro advanced: signin.aws.amazon.com/signup?request_type=register
2. Correo, contraseña y tarjeta: AWS la pide para verificar identidad.
3. Verificación por teléfono.
4. Espera el correo de confirmación; puede tardar hasta 2 días.

MFA en root y la región us-east-2 protegen la cuenta y ubican los datos

Consola AWS

1. Activa MFA (segundo factor) en root (dueño de la cuenta).
2. Elige US East (Ohio), us-east-2: ubicación de los datos.
3. Debes ver: cuenta, MFA y región.
4. Si falla: curso/clase01/cuentas.md.

AWS se habilita por tramos hasta el 17 de noviembre

Calendario

1. Hoy: cuenta creada
2. Clase 2: cuotas y bootstrap (plantilla que crea los permisos base)
3. Clase 3: entorno de trabajo y conexión con Snowflake
4. Clase 4: lake, Glue y datos de muestra
5. Fecha límite: 17 nov
6. Semáforo: rojo verifica · amarillo cuotas · verde claro bootstrap · verde listo

03

DATA LAKE Y LAKEHOUSE

¿Cómo guardamos los tres orígenes tal como llegan, sin imponer un esquema todavía?

Un data lake guarda archivos como objetos en almacenamiento como S3, sin imponer un esquema al escribir

Almacenamiento de objetos: espacio plano de pares clave → bytes, accesible por API (PUT, GET, LIST, DELETE).

s3://lake/

clave

```
bronze/fhvhv/mes=2024-01/fhvhv_tripdata_2024-01.parquet
bronze/taxi_zones/taxi_zone_lookup.csv
bronze/clima/mes=2024-01/open_meteo_2024-01.json
```



Ventajas: capacidad elástica, bajo costo por GB, independiente del cómputo

Límites: sin transacciones ni índices; listar millones de objetos es lento

Las carpetas son parte del nombre de la clave: mes=2024-01/ no es un directorio real.

Un objeto se reemplaza completo, no se edita; no hay transacciones ni índices.

El formato decide cuánto se lee: Parquet, solo lo pedido; CSV y JSON, todo

CSV (zonas)

– Texto por fila

– Sin tipos

– Se lee y parsea todo el archivo

JSON (clima)

– Texto anidado; el esquema se descubre al leer

– Se lee y parsea todo el archivo

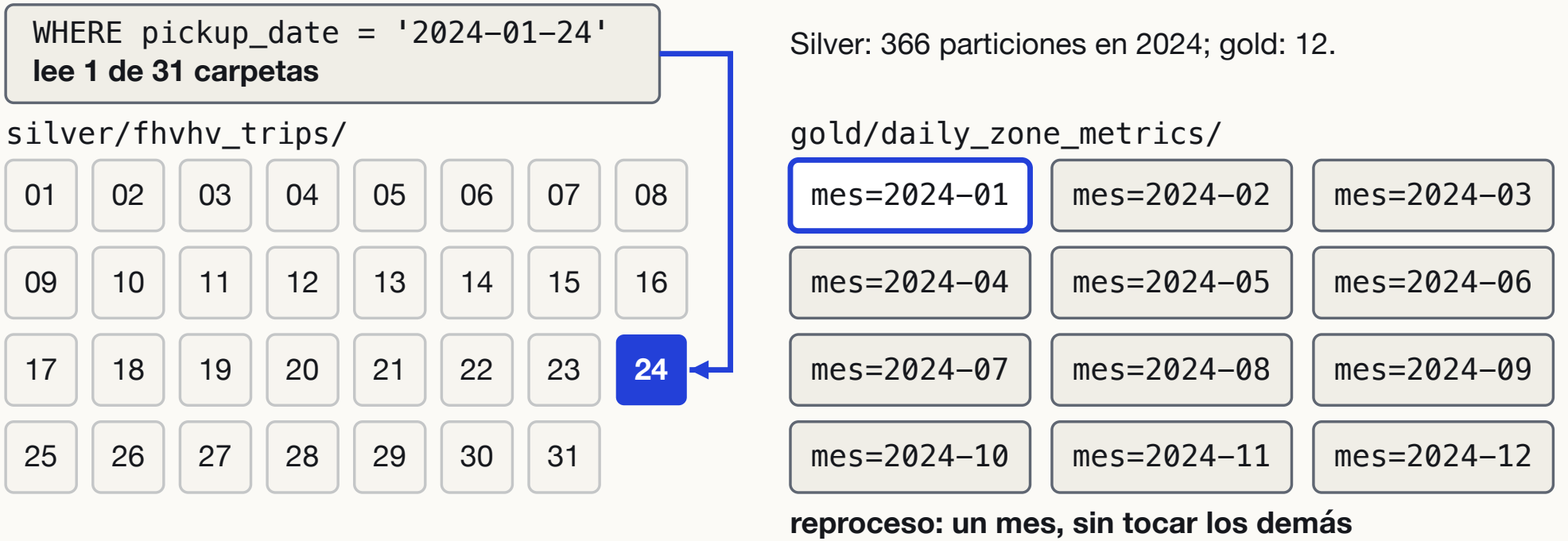
Parquet (viajes)

– Binario columnar

– Con esquema

– Lee solo las columnas pedidas y salta bloques con min/max

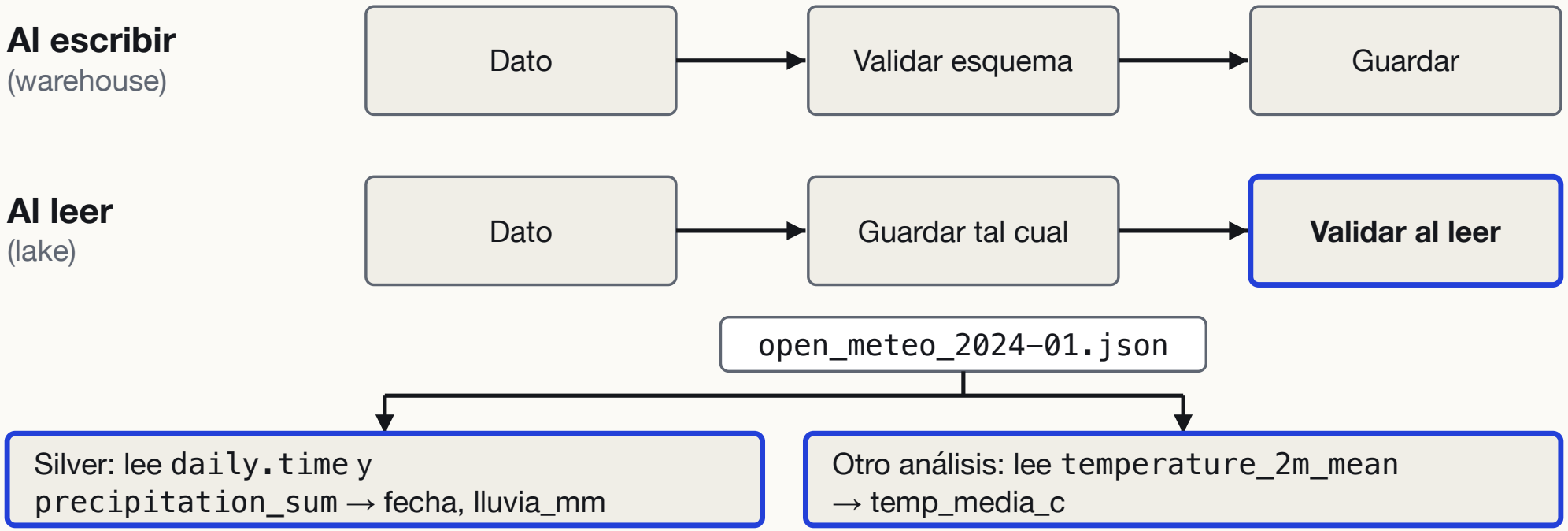
Particionar por fecha permite leer solo las carpetas que la consulta necesita



> Elige una columna que se filtre siempre y tenga pocos valores: por hora serían miles de archivos pequeños.

Límite: demasiadas particiones pequeñas encarecen listar y abrir archivos.

Esquema al leer: el archivo se guarda tal cual y cada consulta le impone su estructura



> Un dato mal formado se descubre al leer, no al guardar.

Parquet trae su esquema; el esquema al leer pesa sobre todo en CSV y JSON.

Sin catálogo ni gobierno, un lake se degrada en un pantano de datos que nadie sabe interpretar

Sin catálogo ni gobierno

```
datos_final.csv
                    clima_nuevo.json
          datos_final_v2.csv
viajes (copia).parquet
```

¿qué esquema?

¿quién lo creó?

¿se puede borrar?

Con catálogo y gobierno

Tabla	Esquema	Ubicación	Partición	Dueño
`fhvhv_trips`	16 columnas	`silver/fhvhv_trips/`	`pickup_date`	equipo de datos
`daily_zone_metrics`	8 columnas	`gold/…`	`mes`	equipo de datos

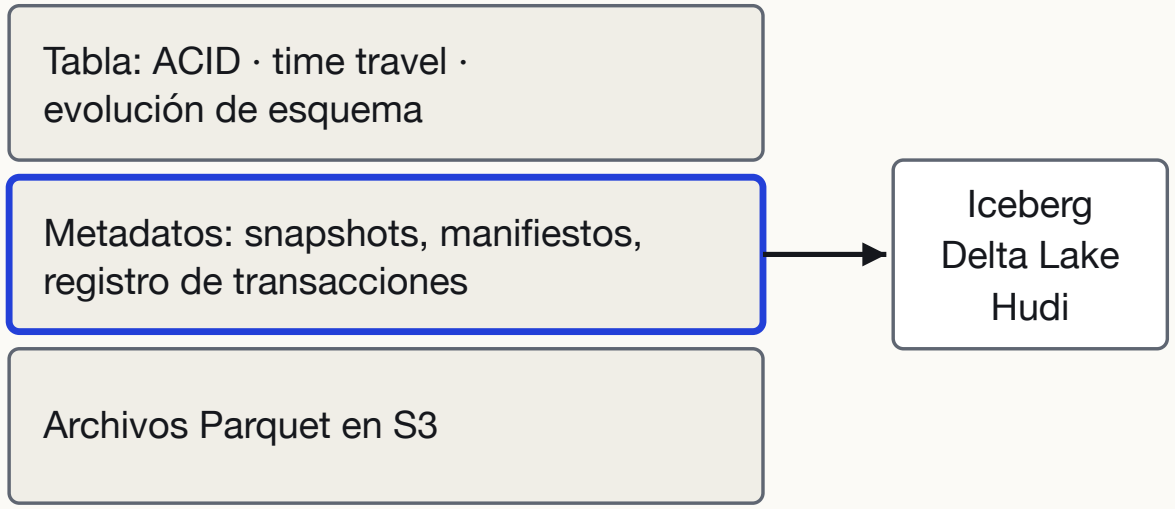
Permisos por prefijo · linaje · calidad

> En el curso: base bigdata_lake en Glue Data Catalog, que registra esquema, ubicación y dueño de cada tabla.

Catálogo: tabla, esquema, ubicación y partición registrados; en el curso, Glue Data Catalog.

Un formato de tabla abierto agrega transacciones ACID y versiones sobre archivos Parquet en el lake

Lakehouse: lake con una capa de metadatos transaccional que da propiedades de tabla de warehouse.



ACID: una escritura se completa entera o no ocurre, y ningún lector ve estados intermedios.

- Dos escritores sobre enero: se ve el snapshot 12 o el 13, nunca archivos a medias
- Time travel: leer gold como estaba ayer

Por qué surge: el lake plano no escribe de forma atómica y copiar al warehouse duplicaba el dato.

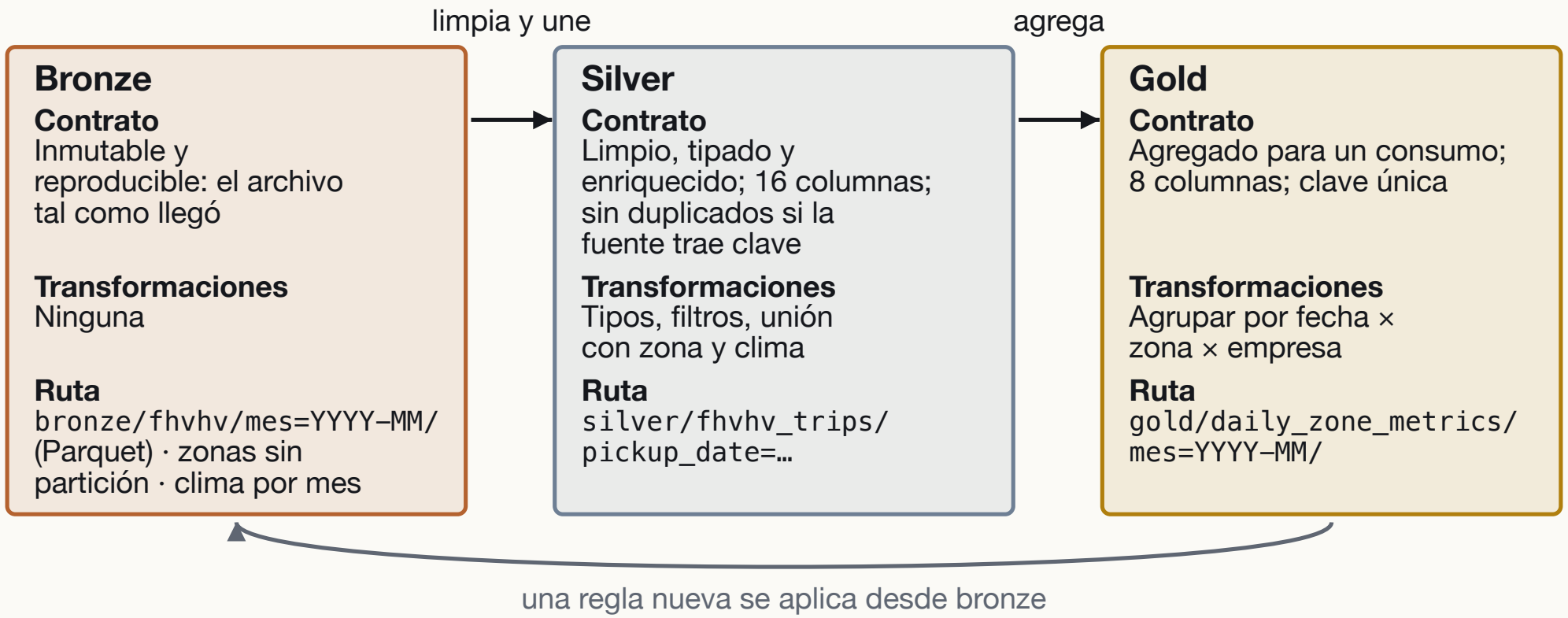
Iceberg, Delta Lake y Hudi. Costo: mantenimiento de metadatos y compactación.

04

MEDALLION Y ARQUITECTURA

¿Cómo organizamos el lake para que cada paso se verifique y se pueda repetir?

Medallion divide el lake en tres capas; cada una promete un contrato verificable



Las capas no se saltan: gold se calcula desde silver, silver desde bronze.

Una fila de bronze llega tal como la entregó la fuente

bronze · ejemplo con el esquema real

hvfhs_license_num	pickup_datetime	PULocationID	trip_miles	trip_time	tips
HV0005	2024-01-24 07:04:00	132	16.2	2460	null

Código, no nombre

Número, no zona

Segundos

Sin valor

En silver el mismo viaje tiene tipos, empresa, zona y lluvia

silver · ejemplo con el esquema real

empresa	pickup_date	duracion_min	propina	zona	lluvia_mm
Lyft	2024-01-24	41.0	0.0	JFK Airport	3.5

HV0005→Lyft | Partición | 2460 s ÷ 60 | null→0 | PULocationID 132 | Por fecha

En gold, los 1 965 viajes de Lyft en JFK ese día son una sola fila

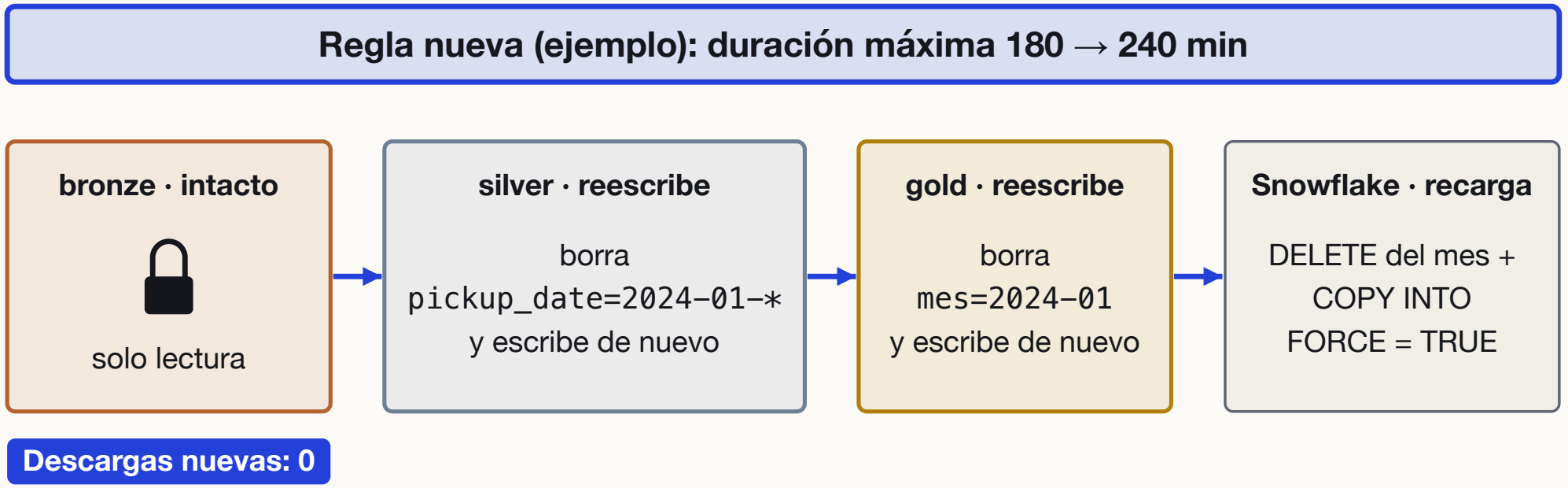
Columna	Valor	Qué es
fecha	2024-01-24	día del viaje
zona	JFK Airport	zona de recogida
empresa	Lyft	HV0005 → Lyft
viajes	1 965	conteo de viajes del grupo
ingreso_total	153 097.03	suma de lo pagado (USD)
propina_promedio	4.05	promedio por viaje
distancia_promedio	16.61	millas por viaje
lluvia_mm	3.5	lluvia del día en NYC

Silver filtra viajes y valida; si la validación falla, se detiene

```sql
SELECT hvfhs_license_num, pickup_datetime, PULocationID,
       trip_miles, trip_time / 60 AS duracion_min
FROM bronze.fhvhv_trips
WHERE pickup_datetime IS NOT NULL
  AND PULocationID IS NOT NULL
  AND trip_miles > 0 AND trip_miles < 100
  AND base_passenger_fare > 0
  AND trip_time / 60 BETWEEN 1 AND 180;
```

Equivalente SQL de limpiar_viajes: un viaje de 0 millas se descarta aquí. Validación: si tras filtrar quedan 0 filas o hay claves duplicadas en gold, quality.py lanza QualityError y el pipeline se detiene.

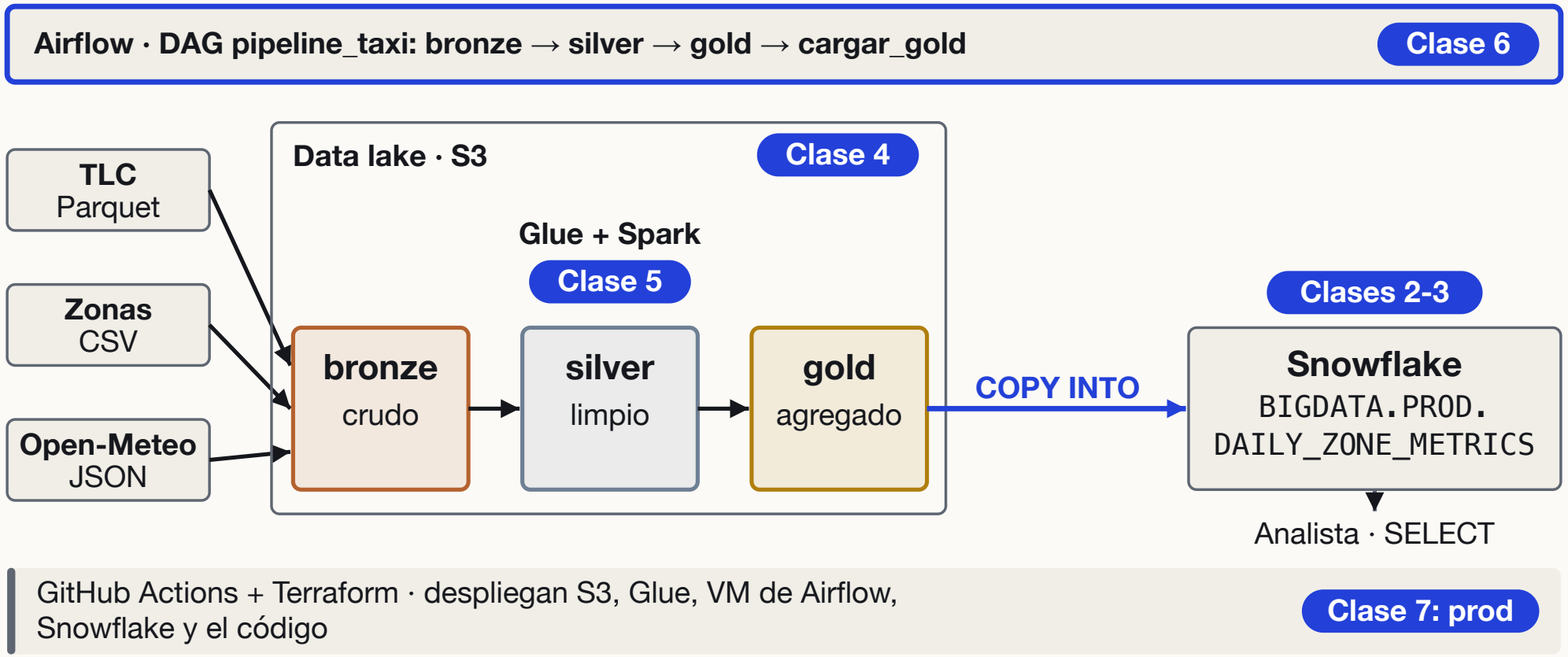
Con bronze inmutable, cambiar una regla reescribe silver y gold sin descargar de nuevo



> Sin bronze inmutable, la regla nueva no se puede aplicar al dato original:
> habría que volver a descargar

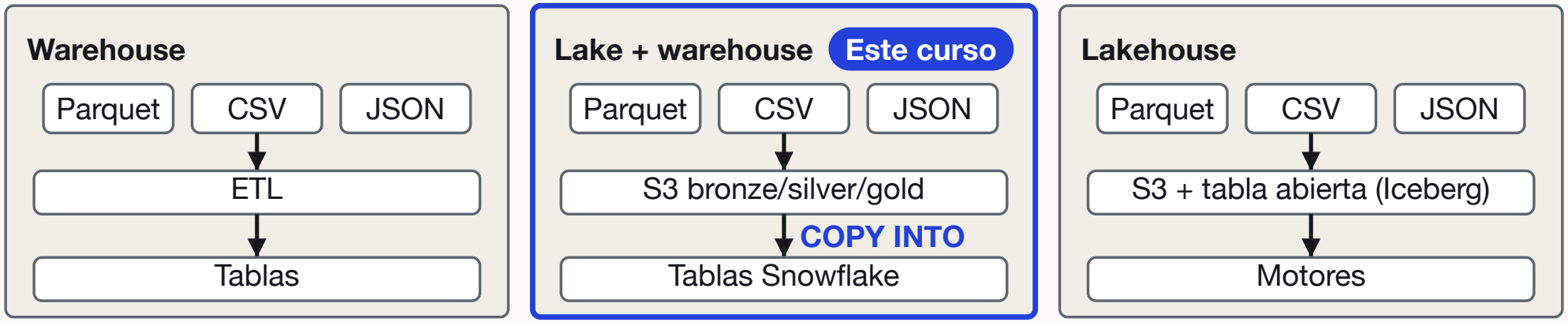
Reproceso por mes: se borra la partición del mes y se reescribe, también en Snowflake.

La arquitectura del curso: lake en S3 con tres capas, Spark en Glue, Airflow y Snowflake como servicio



GitHub Actions y Terraform despliegan la infraestructura y el código.

El curso es lake más warehouse, no lakehouse: así se ve dónde vive cada dato y qué cuesta



Esquema	antes de guardar	al leer y antes de servir	en la tabla
ACID	en el warehouse	solo en Snowflake	en el formato de tabla
Copias de gold	1	2: S3 y Snowflake	1

> Snowflake es transaccional: DELETE + COPY INTO pueden ir en BEGIN…COMMIT

> Por qué aquí: en S3 escribe un solo job por mes y nadie consulta S3 directamente; separar niveles obliga a justificar dónde vive cada dato.

Alternativa descartada: Iceberg en S3 leído desde Snowflake.

DEMO EN VIVO

¿La lluvia cambia los viajes en NYC?

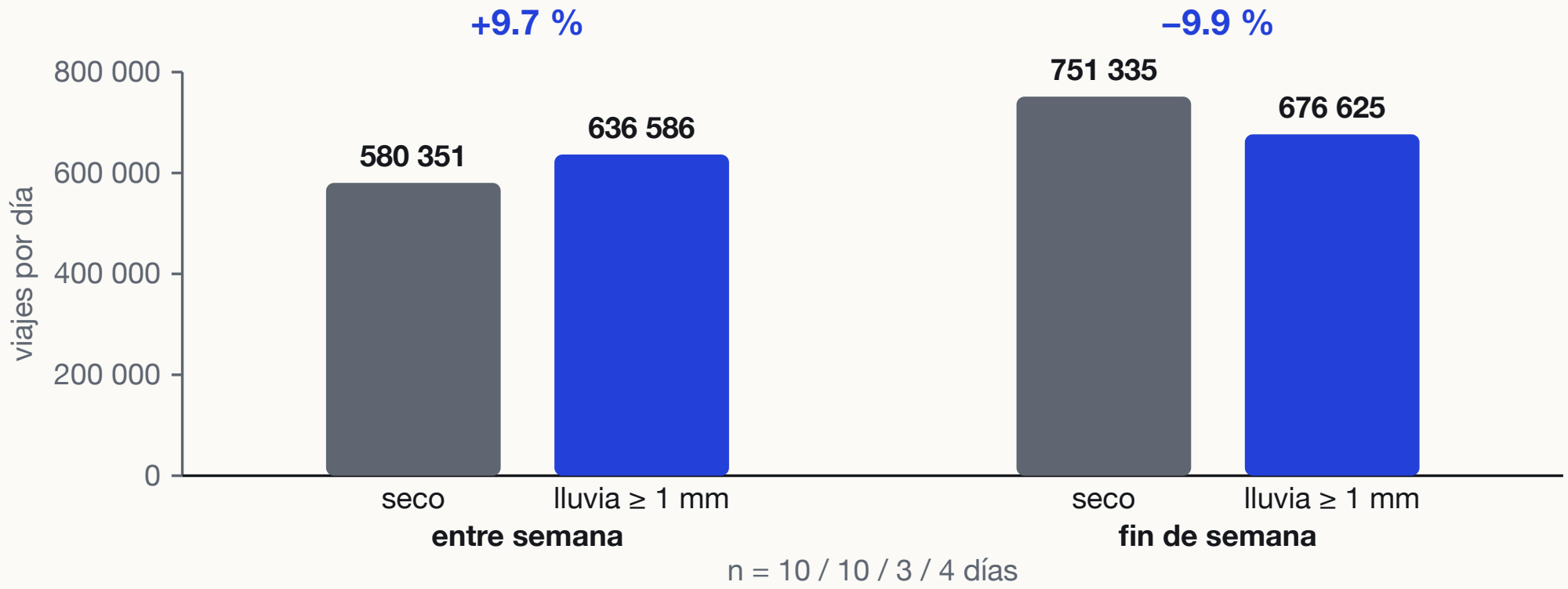
Gold en Snowflake (BIGDATA.PROD) · una consulta · 15 909 filas

La consulta reduce gold a un valor diario y compara días secos con lluvia ≥ 1 mm

```sql
WITH dia AS (SELECT FECHA, MAX(LLUVIA_MM) AS LLUVIA_MM,
                    SUM(VIAJES) AS VIAJES
             FROM BIGDATA.PROD.DAILY_ZONE_METRICS GROUP BY FECHA)
SELECT IFF(DAYOFWEEKISO(FECHA) >= 6,
           'fin de semana', 'entre semana') AS SEMANA,
       IFF(LLUVIA_MM >= 1, 'lluvia ≥ 1 mm', 'seco') AS TIPO_DIA,
       COUNT(*) AS DIAS, ROUND(AVG(VIAJES)) AS VIAJES_PROM_DIA
FROM dia
WHERE LLUVIA_MM = 0 OR LLUVIA_MM >= 1   -- umbral 1 mm; llovizna fuera
GROUP BY 1, 2 ORDER BY 1, 2;
```

Un valor de clima por día para toda la ciudad, por eso MAX. Seco = 0 mm; lluvia = 1 mm o más.

En enero de 2024, la lluvia coincide con +9.7 % de viajes entre semana y –9.9 % en fin de semana



Un valor de lluvia por día para toda la ciudad: no se puede ver el efecto por zona.
4 días de llovizna (0–1 mm) excluidos.

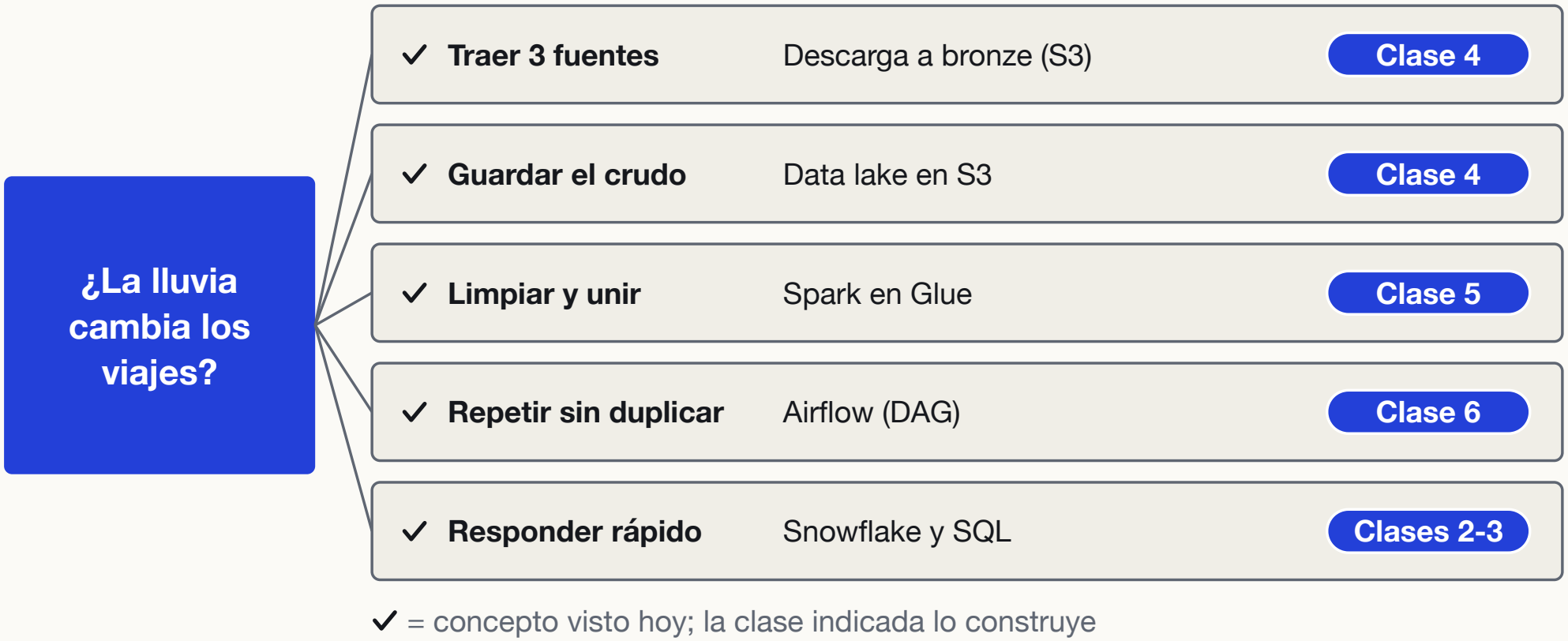
Enero 2024 (31 días) · correlación, no causa · seco = 0 mm · lluvia ≥ 1 mm

15 909

filas leídas en gold, no los 19.7 M viajes del crudo (Query Profile, TableScan). Con 12 meses, el crudo son 239.5 M.

Snowsight · Query Profile

Con 12 meses la comparación gana rigor, pero sigue siendo observacional: la arquitectura la hace repetible



Clase 2: motor de Snowflake. Clase 4: lake. Clase 6: Airflow y Glue.

Tres ideas para llevarse

Ingeniería

– Sistemas repetibles, no consultas sueltas

Arquitecturas

– Warehouse, lake y lakehouse resuelven problemas distintos; el curso combina lake + warehouse

Medallion

– Separa lo que llegó, lo limpio y lo agregado: reprocesa sin perder el origen

El formulario registra tu fork, región y semáforo

Navegador

1. Abre el formulario: enlace en el canal del curso
2. Usuario de GitHub y URL del fork.
3. Pega el resultado de `SELECT CURRENT_REGION();`
4. Marca tu color del semáforo de AWS.
5. Cuenta verificada: pide las cuotas.